

DAY 2 · 화요일

# AI 에이전트 원리 + 클로드 코드

에이전트의 원리와 리서치 에이전트를 이해하고, 과정 전반에 쓸 도구를 구축한다

담당 세션 (3)

## 오전 특강

10:00-11:30 · 90 분

AI 에이전트 원리 + 리서치

## 오후 특강 1

13:00-14:20 · 80 분

Python & 클로드 코드 설치

## 오후 특강 2

14:30-16:00 · 90 분

클로드 코드 설정 + 플러그인

# 전체 과정 로드맵

## DAY 1

월요일

LLM 원리와 이해

## DAY 2

화요일

에이전트 원리 + 클로드 코드

● 진행 중

## DAY 3

수요일

미니 프로젝트 + 리서치 이론 · 팀

## DAY 4

목요일

팀 리서치 — 기획 · 구현 · 실행

## DAY 5

금요일

실행 · 분석 + 보고서 · 발표

진행 단계 : 이론 → 도구 → 실전 → 결과 → 발표

## 세션 구성 (3)

1

### AI 에이전트 원리와 이해 + 리서치

10:00-11:30

오전 특강 · agent = LLM+tools+loop / 리서치 에이전트 개요

90 분

2

### Python & Claude Code 설치

13:00-14:20

오후 특강 1 · Python(python.org) · Claude Code 설치 · 인증

80 분

3

### 클로드 코드 설치 및 주요 설정 (2)

14:30-16:00

오후 특강 2 · 기본 설정 · oh-my-claudecode 플러그인 · 워크플로우

90 분

오전 특강 10:00-11:30 · 90 분

# AI 에이전트 원리와 이해 + AI 에이전트 기반 리서치

---

에이전트의 정의 · 구조 · 자율성과 , 리서치 에이전트의 전체 구조를 이해한다 .

# AI 에이전트 원리와 이해 + AI 에이전트 기반 리서치

## 이 세션에서 다룰 내용

1 AI 에이전트란 무엇인가

2 LLM vs 에이전트

3 에이전트 핵심 루프

4 도구 (tools)

5 메모리와 컨텍스트

6 자율성의 스펙트럼

7 바이트코딩 개념

8 바이트코딩 워크플로우

9 사람의 역할 변화 · 주의점

10 리서치 에이전트 개요

# 이 세션에서 다룰 것

---

- 에이전트의 정의와 구조  
LLM + 도구 (tools) + 반복 (loop)
- 바이트코딩 작업 방식  
자연어로 만들고, 검증하며 반복
- 리서치 에이전트의 큰 틀  
이번 주 팀 프로젝트의 형태

# AI 에이전트란 ?

---

- 정의

LLM( 두뇌 ) + 도구 (tools) + 반복 루프 (loop) 로 목표를 수행

- LLM 과의 차이

답만 내는 게 아니라 ' 행동 ' 하고 결과를 보고 다음 행동을 결정

- 예

검색 → 읽기 → 코드 실행 → 수정을 스스로 반복

# LLM vs 에이전트

## LLM 단독

- 입력→출력 한 번
- 도구 없음
- 기억 없음
- 사용자가 매번 지시

## 에이전트

- 목표를 주면
- 도구를 골라 실행
- 상태·기억 유지
- 스스로 반복·판단

# 에이전트의 동작 사이클



**NOTE** 목표를 달성할 때까지 이 루프를 반복한다

# 도구 (tools) — 에이전트의 손과 발

---

- 도구란

에이전트가 호출하는 외부 기능 — 웹검색 · 파일 · 코드실행 · API

- 왜 필요한가

LLM 의 한계 ( 최신성 · 계산 · 실행 ) 를 실제 행동으로 보완

- 동작 방식

LLM 이 ' 어떤 도구를 어떤 인자로 ' 호출할지 결정 → 결과를 다시 입력

# 메모리와 컨텍스트

---

- 단기 ( 컨텍스트 )

현재 작업 맥락 — 컨텍스트 윈도우 안에 유지

- 장기 ( 외부 메모리 )

파일 · DB · 벡터스토어에 저장해 필요할 때 검색

- 왜 중요한가

긴 작업의 일관성 유지 · 맥락 한계 극복

# 자율성의 정도

---

- 보조형 (copilot)

사람이 매 단계 확인 · 승인

- 반자율

일상 작업은 자동, 중요한 결정만 사람이 확인

- 자율형

목표만 주면 스스로 — 강력하지만 검증 · 안전장치 필수

# 바이브코딩 (Vibe Coding)

---

- 정의

코드를 직접 타이핑하지 않고, 자연어로 의도를 전달해 만든다

- 사람의 일

무엇을 · 왜를 명확히 + 결과를 검증하고 방향 제시

- 핵심

빠른 반복 + 반드시 검증 (출력을 그대로 신뢰 금물)

# 바이브코딩 사이클



**NOTE** 큰 한 번보다, 작은 단위로 반복하는 것이 핵심

# 사람의 역할 변화 · 주의점

---

- 작성자 → 디렉터 · 검수자

타이핑보다 '무엇을 만들지'와 '맞는지' 판단

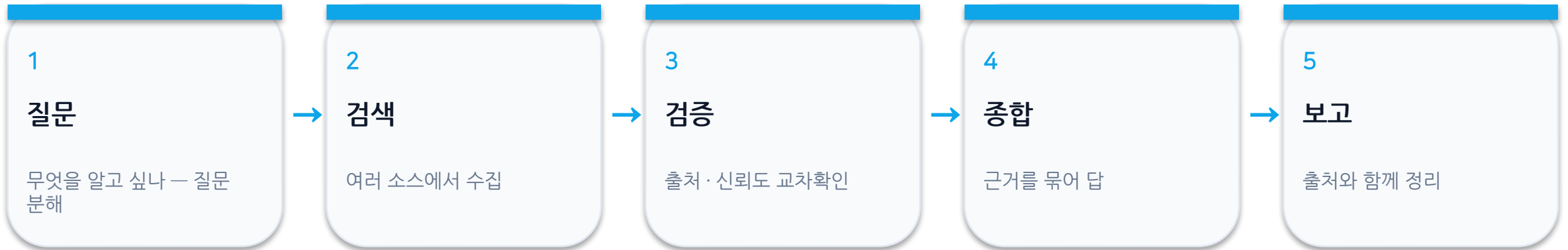
- 검증을 습관화

동작 · 정확성을 직접 확인 — AI는 틀릴 수 있다

- 오늘 오후 실습에서 체험

Claude Code로 직접

# 리서치 에이전트 개요



**NOTE** 자세한 이론은 Day 3 — 오늘은 큰 틀만

오후 특강 1 13:00-14:20 · 80 분

# Python & Claude Code 설치

---

프로젝트용 Python 과 실습 도구 Claude Code 를 각자 노트북에 설치하고 동작을 확인한다 .

# Python & Claude Code 설치

이 세션에서 다룰 내용

1 설치 전 체크리스트

2 Python 설치 (Win/Mac)

3 확인 & 트러블슈팅

4 Claude Code 설치 (Win/Mac)

5 로그인 & 첫 실행

6 설치 실습

# 설치 전 체크리스트

---

- 오늘 설치할 것

Python ( 프로젝트용 ) + Claude Code ( 도구 )

- 대상 OS

Windows / macOS 만 ( Win=PowerShell · Mac=Terminal )

- 계정

Claude 구독 ( Pro/Max ) 또는 Anthropic Console 계정

# python.org 공식 설치 — OS 별

## Windows

- python.org 설치 파일 다운로드
- ★ 'Add python.exe to PATH' 체크
- 확인 : `python --version`
- 터미널 : PowerShell

## macOS

- .pkg 다운로드 → 설치
- ★ 'Install Certificates' 실행
- 확인 : `python3 --version`
- 터미널 : Terminal(zsh)

# 설치 확인 & 트러블슈팅

- 확인

Win: `python --version` · `pip --version` / Mac: `python3` · `pip3 --version`

- Windows 문제

PATH 미설정 → 설치 관리자 'Modify' 에서 PATH 추가 후 터미널 재시작

- macOS 문제

SSL 오류 → 'Install Certificates.command' 실행 · 명령은 `python3` 사용

**NOTE** 대안 (선택): `uv` — 빠른 모던 도구 · `Anaconda` — 데이터 패키지 일괄

# Claude Code 설치 — OS 별

## Windows (PowerShell)

- irm https://claude.ai/install.ps1 | iex
- 또는 npm i -g @anthropic-ai/claude-code
- WSL 없이 네이티브로
- 확인 : claude --version

## macOS (Terminal)

- curl -fsSL https://claude.ai/install.sh | bash
- 또는 brew install --cask claude-code
- 자동 업데이트
- 확인 : claude --version

# Claude Code 로그인 & 첫 실행

---

- 실행

터미널에서 `claude` 입력 → 브라우저 OAuth 로그인

- 인증 방식

Claude 구독 로그인 또는 Console(API 과금)

- 브라우저가 안 열리면

터미널에서 `c` 눌러 URL 복사 → 직접 붙여넣기

# Python & Claude Code 설치 실습

## 실습 단계 (placeholder)

- 1 Python 설치 (Win: PATH 체크 / Mac: 인증서 실행)
- 2 확인 : `python(3) --version` · `pip(3)`
- 3 Claude Code 설치 (Win: PowerShell / Mac: `curl`·`brew`)
- 4 `claude` 실행 → 로그인 → `/help` 확인

## 산출물

Python + Claude Code 실행 · 로그인 완료

문제 시 진단 : `claude doctor`  
( 설치 완료 화면 캡처 삽입 )

오후 특강 2 14:30-16:00 · 90 분

# 클로드 코드 설치 및 주요 설정 (2) — 설정 & 플러그인

---

기본 설정과 CLAUDE.md 를 잡고 , oh-my-claudecode 플러그인으로 작업 환경을 확장한다 .

# 클로드 코드 설치 및 주요 설정 (2) — 설정 & 플러그인

이 세션에서 다룰 내용

- 1 무엇을 설정하나
- 2 기본 설정 (settings · 권한 · 모델)
- 3 CLAUDE.md
- 4 oh-my-claudecode 플러그인
- 5 핵심 커맨드
- 6 설치 & 샘플 실습

# 무엇을 설정하나

## 기본 설정

- 모델 · effort
- 권한 (allow/ask/deny)
- CLAUDE.md ( 프로젝트 규칙 )
- 슬래시 커맨드

## 플러그인 ( 확장 )

- oh-my-claudecode
- 멀티 에이전트 오케스트레이션
- /plugin install
- /autopilot · /team 등

# settings.json & 권한

---

- **설정 파일 위치**

~/ .claude/settings.json ( 전역 ) · .claude/settings.json ( 프로젝트 )

- **권한 (permissions)**

ask( 기본 ) · allow( 미리 허용 ) · deny( 차단 ) — 예 : "Bash(git \*)"

- **모델 · 커맨드**

/model 로 모델 변경 · /config · /help · /clear

# CLAUDE.md — 프로젝트 규칙

---

- **무엇**

매 세션 자동 적용되는 프로젝트 규칙 · 명령어 · 컨벤션

- **생성**

/init 로 자동 생성한 뒤 편집

- **위치**

프로젝트 루트 ./CLAUDE.md (전역은 ~/.claude/CLAUDE.md)

# oh-my-claudecode 란 ?

---

- **무엇**

Claude Code 용 멀티 에이전트 오케스트레이션 플러그인

- **왜**

전문 에이전트들을 조율해 복잡한 작업을 자동화 — 'zero learning curve'

- **구성**

여러 슬래시 커맨드 + 전문 에이전트 (executor·planner·reviewer 등 )

**NOTE** [github.com/yeachan-heo/oh-my-claudecode](https://github.com/yeachan-heo/oh-my-claudecode)

# oh-my-claudecode 설치

---

- ① 마켓플레이스 추가

`/plugin marketplace add https://github.com/Yeachan-Heo/oh-my-claudecode`

- ② 설치 & 셋업

`/plugin install oh-my-claudecode → /omc-setup`

- 사전 준비

team 기능엔 tmux 필요 (brew/apt install tmux)

# 핵심 커맨드 둘러보기

---

- **/autopilot "..."**

아이디어 → 동작 코드까지 자율 실행

- **/team · /ralph · /ultrawork**

다중 에이전트 협업 · 완료까지 지속 · 병렬 처리

- **/plan · /ask**

전략 기획 · 외부 모델 (codex·gemini) 로 교차검토

# 설정 & 플러그인 실습

## 실습 단계 (placeholder)

- 1 기본 설정 (settings.json · 권한)
- 2 /init 로 CLAUDE.md 생성 · 편집
- 3 oh-my-claudecode 설치 (/plugin → /omc-setup)
- 4 샘플 작업 1 회전 (예 : /autopilot 간단 작업)

## 산출물

설정 완료 + 플러그인 설치된 작업 환경

예시 : examples/claude-setup  
(settings.json · CLAUDE.md)

# 핵심 정리 및 다음 과정

## 핵심 정리

- 에이전트 = LLM + tools + loop, 자율성에는 스펙트럼이 있다
- 리서치 에이전트 = 질문→검색→검증→종합→보고
- Python + Claude Code 설치, oh-my-claudecode 플러그인으로 작업 환경 완성

## 다음 과정 — Day 3

세팅한 도구로 실생활 미니 프로젝트를 기획 · 개발하고, 리서치 프로세스 이론을 배운 뒤 팀과 주제를 정합니다.